

Figure 3.25 The paths now have the data from their parent element bound to them and respond accordingly when a discrete color scale based on region is applied.

One drawback with this method is that the paths can't take advantage of the D3 `.insert()` method's ability to place the elements behind the labels or other visual elements. To get around this, we'll need to either append icons to `<g>` elements that have been placed in the proper order, or use the `parentNode` and `appendChild` functions to move the paths around the DOM like we described earlier in this chapter.

The other drawback is that because these paths were added using `cloneNode` and not `selection#append` syntax, they have no data bound to them. We looked at rebinding data back in chapter 1. If we select the `<g>` elements and then select the `<path>` element, this will rebind data. But we have numerous `<path>` elements under each `<g>` element, and `selectAll` doesn't rebind data. As a result, we have to take a more involved approach to bind the data from the parent `<g>` elements to the child `<path>` elements that have been loaded in this manner. The first thing we do is select all the `<g>` elements and then use `.each()` to select all the path elements under each `<g>`. Then, we separately bind the data from the `<g>` to each `<path>` using `.datum()`. What's `.datum()`? Well, datum is the singular of data, so a piece of data is a datum. The `datum` function is what you use when you're binding just one piece of data to an element. It's the equivalent of wrapping your variable in an array and binding it to `.data()`. After we perform this action, we can dust off our old scale from earlier and apply it to our new `<path>` elements. We can run this code in the console to see the effects, which should look like figure 3.25.

```
d3.selectAll("g.overallG").each(function(d) {
    d3.select(this).selectAll("path").datum(d)
});

var tenColorScale = d3.scale
    .category10(["UEFA", "CONMEBOL", "CAF", "AFC"]);

d3.selectAll("path").style("fill", function(p) {
    return tenColorScale(p.region)
}).style("stroke", "black").style("stroke-width", "2px");
```

Now you have data-driven icons. Use them wisely.

3.4 Summary

Throughout this chapter, we dealt with methods and functionality that typically are glossed over in D3 tutorials, such as the color functions and loading external content like external SVG and HTML. We also saw common D3 functionality, like animated transitions tied to mouse events. Specifically, we covered